

驯服长尾：借助自适应草稿模型的高效推理强化学习训练

Qinghao Hu* Shang Yang* Junxian Guo Xiaozhe Yao Yujun Lin
Yuxian Gu Han Cai Chuang Gan Ana Klimovic Song Han
MIT ETH Zurich NVIDIA MIT-IBM AI Lab

摘要

具备强推理能力的大语言模型通常借助强化学习 (RL) 训练，但其响应生成呈持续的**长尾**分布：少量极长响应主导执行时间，造成资源闲置并推高成本。本文提出 TLT，一套以自适应推测解码无损加速推理 RL 训练的系统。其难点在于负载动态变化、目标模型不断演化，以及草稿模型的训练开销。TLT 包含两个协同组件：(1) **自适应草稿模型**，在长尾生成期间利用空闲 GPU 持续训练轻量草稿模型，以零额外关键路径成本保持其与目标模型对齐；(2) **自适应 Rollout 引擎**，维护节省显存的预捕获 CUDAGraph 池，并为每个输入批自适应选择推测解码策略。评测显示，TLT 相对先进系统带来超过 1.7 $\times$ 的端到端 RL 训练加速，不损失模型准确率，并免费产出可用于高效部署的高质量草稿模型。

1 引言

OpenAI-o1、DeepSeek-R1 和 Gemini 等推理 LLM 在数学、编程和多学科任务上取得显著进展。强化学习已经成为赋予 LLM 强推理能力的标准路径，但其工作负载与传统 RLHF 有本质差别：推理轨迹通常长一个数量级以上。

一次 RL 步骤先在 rollout 阶段生成大量候选回答，再在推理和训练阶段据此更新模型。rollout 常占步骤时间约 85%，且响应长度有顽固的长尾：多数序列很短，少数抵达最大长度，令已完成请求对应的 GPU 长时间闲置。ByteDance 的 32B 训练轨迹显示，即使使用 128 张 GPU，385 步也耗时 11 天；单步平均约 40 分钟。Berkeley、Alibaba 与 AMD 的大规模 RL 系统也报告了相同现象。

量化和稀疏化虽可加速，却会改变模型分布。推测解码 (SD) 由轻量草稿模型提出 token，再由目标模型并行验证；其输出分布与原目标模型严格一致，并能将小批量、显存受限的自回归生成转向更充分利用计算的过程，因而特别适合长尾尾部。把已有 SD 直接放入 RL 仍有三项挑战：**C1** 目标权重持续更新，使草稿模型过时；**C2** 专用草稿模型本身有训练成本；**C3** rollout 的有效批大小随请求完成而波动，静态超参数既低效又可能 OOM。

TLT 的关键观察是：长尾期间，先结束序列释放出的资源形成可利用的“气泡”。系统将这些资源用于可抢占的草稿模型更新，同时根据当下批大小选择 SD 策略，因此不改变原有 on-policy RL 算法。本文贡献为：提出持续对齐的自适应草稿模型、按批大小在线调

参的自适应 rollout 引擎，以及在多模型、多 GPU 平台和不同规模集群上的端到端验证。

2 背景与动机

2.1 推理与强化学习

推理 LLM 的两个典型特征是**长思维链**和**自我反思**：模型用更多测试时生成预算探索推理路径，并能回看、修正自己已生成的内容。这些能力往往在 RL 阶段获得。

本文采用的 GRPO 一步包含三阶段：(1) **rollout**：目标模型为每个提示生成多个候选；(2) **推理**：目标模型和固定参考模型计算 logits 与 KL 惩罚，同时用规则计算奖励；(3) **训练**：由奖励和 KL 项构造损失，更新目标模型。GRPO 用组内基线替代大型 critic，从而降低训练开销。

2.2 为何采用推测解码

草稿模型快速产生候选 token，目标模型一次前向并行验证它们；系统接受直至首次不匹配前的 token，并保留该位置目标模型给出的正确 token。因此，SD 保持与目标模型相同的输出分布。在长尾阶段，活跃请求少、批量变小，SD 可提高每步接受 token 数并提升 GPU 吞吐。

然而，RL 的目标模型不断变化，固定草稿模型接受率会下降；专门训练草稿模型又会额外占用资源；而批大小从大到小快速收缩，要求 SD 策略随负载调整。TLT 分别以自适应草稿模型解决 C1/C2，以自适应调度解决 C3。

3 TLT 概览

系统遵循四项原则：**无损**，不引入异步陈旧性或有损近似；**不干扰**，草稿训练是可抢占的 spot 任务；**自动且简单**，无需用户手调草稿模型和 SD 参数；**可泛化、可扩展**，适用于 GRPO、RLOO 等推理 RL 算法及不同模型、集群规模。目标则是缩短长尾、提高集群利用率，并免费得到可部署的草稿模型。

rollout 引擎保存目标模型与草稿模型的预捕获 CUDAGraph，并由自适应 SD 管理器选择每批策略。它记录接受长度与步骤延迟，将这些在线反馈给调参器。与此同时，集中式 Worker Coordinator 监测 rollout worker；一旦长尾阶段出现足够空闲 GPU，便启动

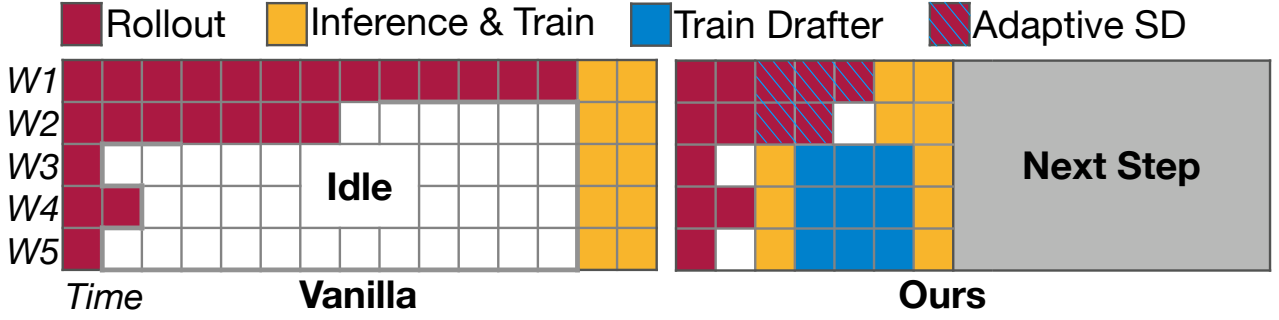


图 1: 推理 RL 中长尾 rollout 与工作负载不均衡的示意; TLT 通过自适应草稿模型加以缓解。

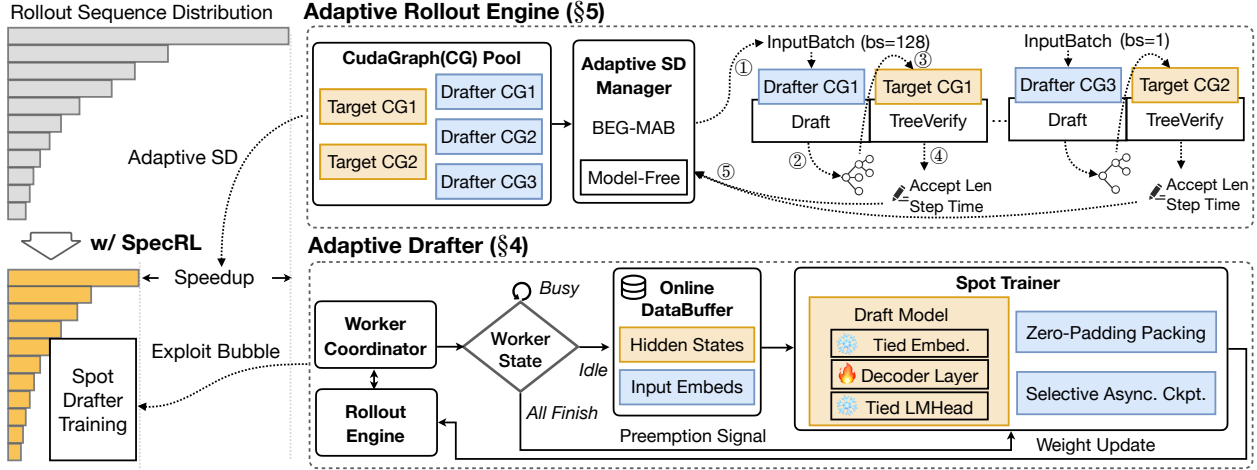


图 2: TLT 架构与工作流。自适应 rollout 引擎选择草稿与目标 CUDAGraph; 空闲 worker 被协调器临时用于训练草稿模型。

Spot Trainer。训练复用当前和历史 rollout 缓存的隐藏状态与 token，支持随时中断、恢复。两者结合可在不降低质量的前提下获得加速。

4 自适应草稿模型

4.1 草稿模型

TLT 使用仅含一层 **Transformer decoder** 的轻量模型模仿目标 LLM 的 token 生成行为。它复用目标模型的 Embedding 与 LM Head，并在训练中冻结两者，只更新一层 decoder，参数量约为目标模型的 $1/\text{layer_num}$ 。这不同于直接使用一个较小但仍有多层的同家族 LLM：例如 Qwen2.5-32B 的 0.5B 版本虽参数少得多，仍含 24 层、顺序计算延迟不可忽略。实验中，一层草稿模型以近似参数量取得 $2.4\times$ 更低延迟。

训练框架直接利用 RL 工作流产生的数据。目标模型预填充时，从特定层收集隐藏状态；它们与输入 token embedding 拼接并经线性降维后，输入草稿 decoder。因为目标模型远大于草稿模型，重新预填充远比一次草稿训练昂贵；系统将本来就会产生的隐藏状态缓存到主机内存并复用。框架支持隐藏状态 L1 损失、输出 logits 的交叉熵损失，或二者组合。EAGLE、HASS 和 EAGLE-3 等策略都可在此表达；本文默认 EAGLE，因为它以较低训练成本提供较高接受长度和更快收敛。

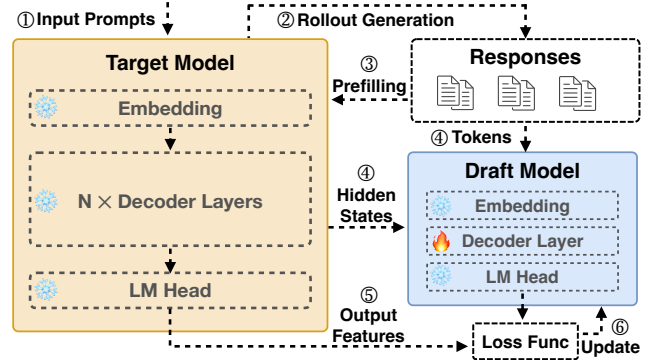


图 3: 草稿模型训练：复用目标模型隐藏状态的统一流程，以及不同训练策略的配置。

4.2 Spot Trainer

Worker Coordinator 连接 Rollout Engine 与 Spot Trainer。在系统中，一个 worker 是一个 rollout 实例；例如 TP=8 的完整 DGX-H100 节点即一个 worker。基于 ZeroMQ 的集中式协调器维护全局状态，worker 在 *BUSY*（服务 rollout）、*IDLE*（推理结束且显存已释放）和 *TRAINING*（更新草稿模型）之间切换并报告状态。空闲 worker 数超过阈值时，首个合格 worker 建立训练会话，同一数据并行组的其他 worker 可加入；rollout 完成时立即优雅停止训练。

普通做法把草稿训练串行加在 rollout、KL 推理和目标训练之后，即便复用部分预填充，仍会扩大关键路

Algorithm 1 分桶 ϵ -贪心 (BEG) MAB 选择器

Require: 策略集 $\mathcal{S}$, 批量阈值 $\mathcal{T}$, 探索率 ϵ , 窗口 w **Ensure:** 选择的策略 s^*

```
1: 按 Tokens_to_Verify 将  $\mathcal{S}$  分组, 并映射到批量桶  $\mathcal{B}$ ;  
2: 为每个策略维护长度为  $w$  的近期奖励和接受长度队列  
3: 对当前批量选出所在桶的候选集合  $V$   
4: if  $|V| = 1$  then  
5:   return 唯一候选  
6: end if  
7: if 随机数  $u < \epsilon$  then  
8:   return  $V$  中随机策略 ▷ 探索  
9: else  
10:  return  $\arg \max_{s \in V} \text{Median}(R_s)$  ▷ 利用  
11: end if
```

径。Spot Trainer 不等待全部响应完成, 而用先完成的部分响应非阻塞训练。为避免早完成序列偏短造成分布失配, **DataBuffer** 跨 RL 步保存推理阶段的隐藏状态和 token, 并采用一步偏移采样: 当前部分数据之外, 优先取前一步的长序列。实验表明, 这一组合与使用完整响应集的效果相当。

Spot Trainer 还采用**选择性异步检查点**: 保存状态交给后台线程, 并只落盘可训练 decoder 参数, 跳过冻结的 Embedding 和 LM Head, 以降低抢占损失; 以及**序列打包**: 把变长序列串接成紧凑序列, 以 attention mask 保持边界, 消除 padding 的计算、通信和显存浪费。

5 自适应 Rollout 引擎

5.1 自适应 SD 管理器

树状草稿不同于单分支线性草稿。草稿模型在每层取概率最高的 topK 个 token, 延展 Draft_Depth 层候选树, 再选取树中置信度最高的 Tokens_to_Verify 个 token 交给目标模型并行验证。这提高了单次验证可接受 token 数。

自适应是必要的。SD 在中等批量也能加速, 但 rollout 开始时批量大、随后因短响应完成而迅速缩小。固定策略在大批量既可能无益, 也可能触发 OOM。管理器持续调节参数: 选择安全策略、学习每类负载的有效配置, 并只在剩余请求数小于阈值 (默认 32) 时启用 SD。

分桶 CUDAGraph 捕获解决策略多样时的显存膨胀。CUDAGraph 要为每种批大小和策略保存静态图, 同时覆盖目标与草稿模型, 朴素做法显存随策略数线性增长。系统采用三项优化: (1) 将批大小分桶, 而非逐一捕获; (2) 分别捕获目标和草稿模型, 因为 topK 只影响草稿端而待验证 token 数只影响目标端; (3) 对共享相同参数的策略合并捕获。这样既减少冗余显存, 又避免尝试明显不适合该批大小的策略。

5.2 自动调参和无模型草稿

BEG 将每个三元组 (Draft_Depth, topK, Tokens_to_Verify) 视为一只 bandit 臂, 奖励为每秒接受 token 数。它先按待验

证 token 数把策略与批量桶对应; 候选不唯一时, 以概率 ϵ 探索, 否则选择滑动窗口中位数奖励最高的策略。近期奖励队列使其能跟随 RL 训练中的非平稳变化。

同一提示的多个 rollout 往往有相似片段, 如数学符号或代码语法。系统还提供非参数化的**无模型草稿器**: 它从每个提示的 rollout 响应动态构建 n-gram 检索库, 按前序 n-gram 预测常见后续 token。学习型草稿器尚不可用或预计低效时, 策略选择器将其作为稳健后备, 从训练初期起维持 SD 加速。

6 评测

6.1 设置

实现建立在 VeRL 与 Ray 之上; 协调器使用 ZeroMQ, 草稿模型沿用 EAGLE 训练流程并使用 FSDP2, 异步检查点使用 PyTorch DCP, rollout 后端为 SGLang。实验默认在 8 台 DGX H100 (共 64 GPU) 上进行, 也覆盖 DGX A100。每台 H100 服务器含 8 张 H100、112 核 CPU、2TB 内存、900GB/s NVLink 和 400Gb/s InfiniBand。

评测模型包括 Qwen2.5-7B、DeepSeek-R1-Distill-Qwen-7B、未经额外微调的 Qwen2.5-32B, 以及 Llama-3.3-70B-Instruct。主 RL 数据来自含可验证数学与编程题的 Eurur-2-RL 子集; 草稿模型预热使用 OpenThoughts2-1M 子集。遵循 GRPO, 温度设为 0.9、最大生成长度为 32K, 按模型规模使用 TP=2/4/8 和 BF16 Adam。比较对象是 Open-R1、VeRL, 以及关闭自适应草稿模型、仅用无模型草稿器的 TLT-Base; 指标为每 RL 步中提示与响应 token 总数除以步骤时长。

6.2 端到端结果

在 H100/A100 与多种模型规模上, TLT 始终优于 VeRL; Open-R1 因无法充分并发利用 GPU, 且 rollout 与训练批量耦合较紧, 表现更弱。TLT-Base 也取得明显收益, 说明无模型草稿器可在早期持续提供加速。Qwen2.5-7B 与 32B 的平均奖励曲线和 VeRL 高度重合, 验证系统没有改变底层 RL 的学习动态或模型质量。

6.3 自适应 SD

仅最大化接受长度并不足以最大化真实速度。网络实验发现, 增大草稿深度通常增加接受长度, 但收益会饱和; 不同批大小的最优待验证 token 数也不同, 大批量应验证更少 token。案例中, Qwen-32B 处理 128 个请求时, TLT 只在剩余请求少于 32 后打开 SD, 并在该区间继续适配配置, 相对无 SD 基线获得 $2.44\times$ 加速。

分桶 CUDAGraph 也显著降低显存: Llama-3-8B、H100 TP=4、四种策略时, 单一策略为 7.81GB, 朴素多策略为 30.39GB, 而分桶方案仅 10.69GB, 即相对朴素方案减少 $2.8\times$ 。在 B200、H100、A100、RTX 5090/4090/3090 上, 单请求 rollout 的 SD 加速分别为 $2.33\times$ 、 $2.61\times$ 、 $2.79\times$ 、 $2.91\times$ 、 $2.87\times$ 和 $3.22\times$, 表明

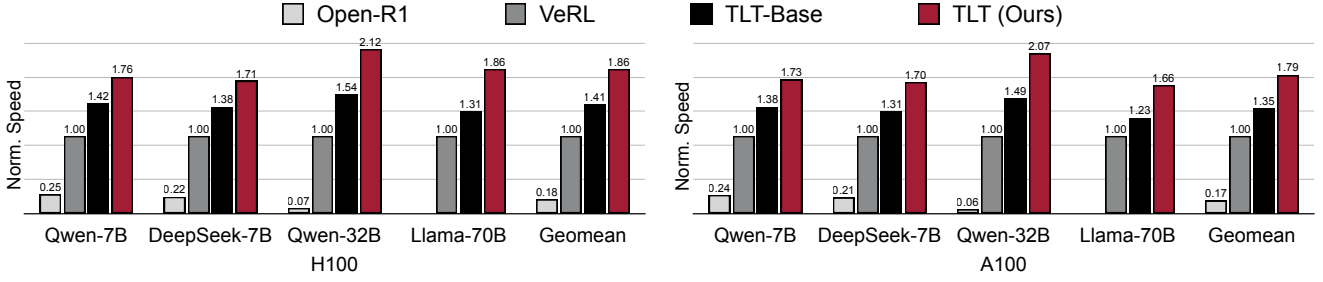


图 4: 端到端训练吞吐。纵轴为运行 GRPO 时相对训练吞吐; TLT 相比 VeRL 达到 1.7–2.1 $\times$ 加速。

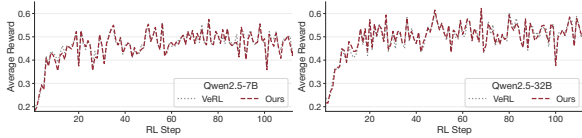


图 5: VeRL 与 TLT 的端到端 RL 平均奖励曲线。

表 1: 不同批大小下 SD 的加速比 (Qwen-32B, H100, TP=4)。

批大小	验证 16	验证 32	验证 48	验证 64
1	3.22 $\times$	3.46 $\times$	3.56 $\times$	3.62$\times$
2	3.08 $\times$	3.28 $\times$	3.39$\times$	3.38 $\times$
8	2.73$\times$	2.63 $\times$	2.51 $\times$	2.27 $\times$
32	2.48$\times$	2.23 $\times$	1.90 $\times$	1.70 $\times$

其跨 GPU 有效。

6.4 Spot Trainer 与草稿器分析

预热后, 自适应草稿器的 top-3 下一 token 准确率持续提高; 每次 RL 更新会因分布变化短暂下降, 但数次草稿训练迭代即可恢复。对 Target-R, 平均接受长度在 RL 训练中由 4.59 提升至 6.53, 在下游任务上由 3.76 提升至 5.15。选择性异步检查点通过后台写入和仅保存可训练参数, 将检查点延迟降低 9.2 $\times$; 序列打包相对普通 batching 提升 2.2 $\times$ 训练吞吐。

HASS 与 EAGLE-3 的接受长度、吞吐略高, 但其“训练时测试”需要多次前向, 训练开销分别约为 EAGLE 的 3 $\times$ 与 7 $\times$, 且更难收敛。由于 rollout 气泡预算有限, 本文选择性能相近、实现更简洁的 EAGLE。OSD 风格的知识蒸馏进一步改善小型 Qwen-0.5B 和 EAGLE 草稿器质量, 说明可与 TLT 默认训练结合。

系统额外开销主要来自草稿权重更新/优化器卸载 (小模型使其少于一步时间的 1%)、从普通解码切换到 SD 时不超过 3 秒的重新预填充, 以及偶发的训练协调竞争。草稿器不必每步更新, 每 10 步训练一次已足以维持准确率, 因此总收益远大于这些开销。

7 讨论

该方法还适用于三类场景: (1) 所有响应都很长时, KV cache 压力限制活跃批量, 天然落入 SD 的高收益区; (2) 多轮工具调用 RL 中, 部分请求在无 GPU 的工

具执行期间保留 KV cache, 减少活跃解码请求; (3) 在线服务与边缘部署中, 固定目标模型可直接使用 TLT 训练出的草稿模型, 并针对变化负载做自适应解码。

本文不通过部分 rollout 异步更新模型来打破 on-policy 同步约束, 因为同一 rollout 混入多个策略版本会使梯度估计偏置、改变学习动态。有限异步若能保持算法正确性, 可与本文 SD 结合, 但长尾问题依然存在。GRPO 之外, RLOO、DAPO、REINFORCE 与 REINFORCE++ 的总体工作流相似, 主要差异在奖励和 KL 正则化, 因而都与自适应草稿器和 spot 训练兼容。

8 相关工作

现有 RLHF 系统如 DeepSpeed-Chat、GEAR、VeRL、RLHFuse 与 ReaLHF 主要优化多模型管理、数据传输、并行方式和设备分配, 却未专门解决推理 RL 的 rollout 瓶颈。允许陈旧响应、提前终止 rollout 或 tail batching 的近期方法可以提速, 但可能损害质量; 尤其相同查询的推理轨迹长度仍高度可变。TLT 保留原 RL 算法, 在无损条件下解决 rollout 长尾。

推测解码已有检索式方法、在目标模型上增加预测头的 Medusa/Hydra、树式 SpecInfer, 以及 EAGLE/HASS 等专用自回归草稿器; 还有面向静态推理的 RSD 与 SpecReason。与这些静态模型设定不同, 本文的重点是目标模型持续更新的动态 RL 训练。

9 结论

TLT 以自适应草稿模型缓解推理 RL 训练中的长尾瓶颈: 训练时跟随目标权重更新, 推理时跟随批大小变化。系统在不牺牲模型质量的条件下显著提升吞吐, 并将高质量草稿模型作为免费副产物产出。

致谢

感谢 Jiarong Xing 教授在论文修改期间的指导, 感谢 ASPLOS 和 SOSOP 的匿名审稿人提出的宝贵意见; 感谢 MIT-IBM Watson AI Lab、MIT AI Hardware Program、MIT Amazon Science Hub、Hyundai Motor Company 与 National Science Foundation 的支持。